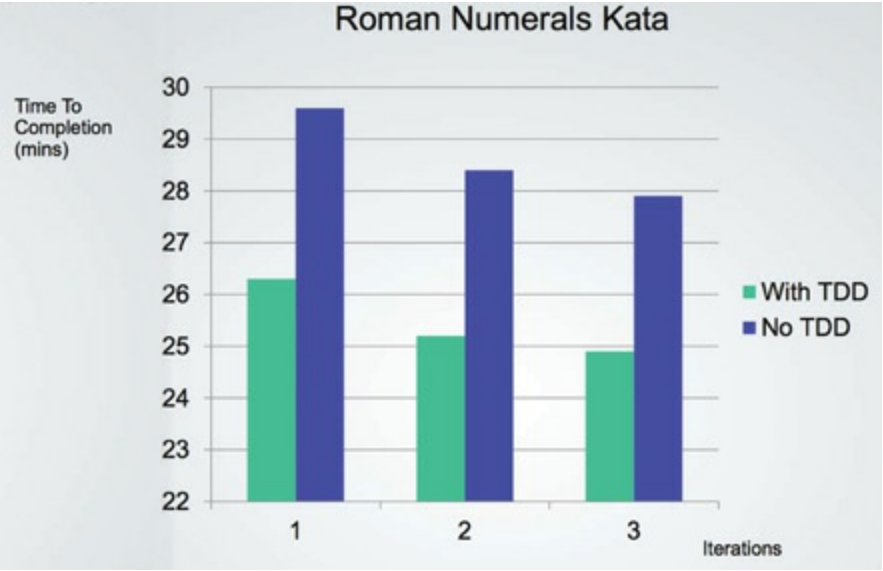

Figure 1.6 Time to completion by iterations and use/non-use of TDD

First, notice the learning curve apparent in Figure 1.6. Work on the latter days is completed more quickly than the former days. Notice also that work on the TDD days proceeded approximately 10% faster than work on the non-TDD days, and that even the slowest TDD day was faster than the fastest non-TDD day.

Some folks might look at that result and think it's a remarkable outcome. But to those who haven't been deluded by the Hare's overconfidence, the result is expected, because they know this simple truth of software development:

> *The only way to go fast, is to go well.*

And that's the answer to the executive's dilemma. The only way to reverse the decline in productivity and the increase in cost is to get the developers to stop thinking like the overconfident Hare and start taking responsibility for the mess that they've made.

The developers may think that the answer is to start over from scratch and redesign the whole system—but that's just the Hare talking again. The same overconfidence that led to the mess is now telling them that they can build it better if only they can start the race over. The reality is less rosy:

> *Their overconfidence will drive the redesign into the same mess as the original project.*

CONCLUSION

In every case, the best option is for the development organization to recognize and avoid its own overconfidence and to start taking the quality of its software architecture seriously.

To take software architecture seriously, you need to know what good software architecture is. To build a system with a design and an architecture that minimize effort and maximize productivity, you need to know which attributes of system architecture lead to that end.

That’s what this book is about. It describes what good clean architectures and designs look like, so that software developers can build systems that will have long profitable lifetimes.

2
A TALE OF TWO VALUES

Every software system provides two different values to the stakeholders: behavior and structure. Software developers are responsible for ensuring that both those values remain high. Unfortunately, they often focus on one to the exclusion of the other. Even more unfortunately, they often focus on the lesser of the two values, leaving the software system eventually valueless.

BEHAVIOR

The first value of software is its behavior. Programmers are hired to make machines behave in a way that makes or saves money for the stakeholders. We do this by helping the stakeholders develop a functional specification, or requirements document. Then we write the code that causes the stakeholder's machines to satisfy

those requirements.

When the machine violates those requirements, programmers get their debuggers out and fix the problem.

Many programmers believe that is the entirety of their job. They believe their job is to make the machine implement the requirements and to fix any bugs. They are sadly mistaken.

ARCHITECTURE

The second value of software has to do with the word "software"—a compound word composed of "soft" and "ware." The word "ware" means "product"; the word "soft"… Well, that's where the second value lies.

Software was invented to be "soft." It was intended to be a way to easily change the behavior of machines. If we'd wanted the behavior of machines to be hard to change, we would have called it *hard*ware.

To fulfill its purpose, software must be soft—that is, it must be easy to change. When the stakeholders change their minds about a feature, that change should be simple and easy to make. The difficulty in making such a change should be proportional only to the scope of the change, and not to the *shape* of the change.

It is this difference between scope and shape that often drives the growth in software development costs. It is the reason that costs grow out of proportion to the size of the requested changes. It is the reason that the first year of development is much cheaper than the second, and the second year is much cheaper than the third.

From the stakeholders' point of view, they are simply providing a stream of changes of roughly similar scope. From the developers' point of view, the stakeholders are giving them a stream of jigsaw puzzle pieces that they must fit into a puzzle of ever-increasing complexity. Each new request is harder to fit than the last, because the shape of the system does not match the shape of the request.

I'm using the word "shape" here in a unconventional way, but I think the metaphor is apt. Software developers often feel as if they are forced to jam square pegs into round holes.

The problem, of course, is the architecture of the system. The more this architecture